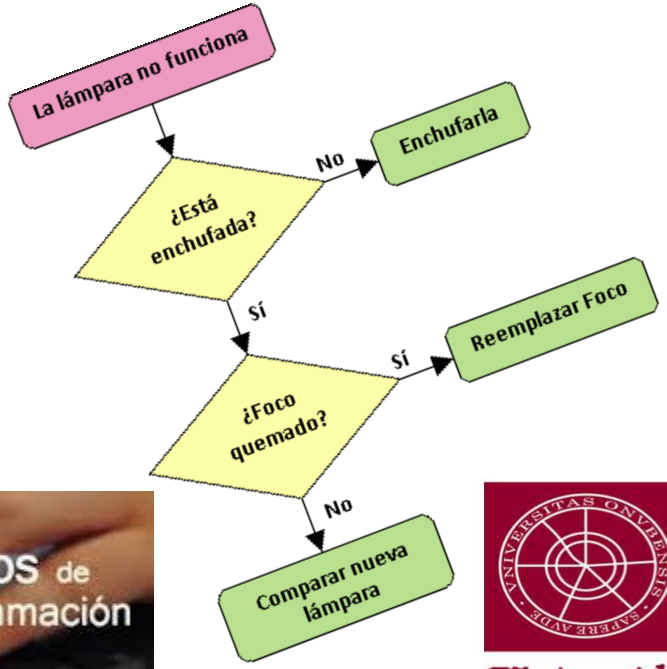
Tema 4.

Tipos Estructurados de Datos

Fundamentos de Programación
Grado en Ingeniería Informática

Profesores:
José Manuel Martín Ramos
Francisco Roche Beltrán.



ETSI ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
DEPARTAMENTO DE TECNOLOGÍAS DE LA INFORMACIÓN
Universidad de Huelva

ÍNDICE

1. Registros. Registros jerarquizados.
2. Tablas. Tablas de registros.
3. Esquemas de recorrido y búsqueda.
4. Cadenas.
5. Objetos y Clases.

1. Registros. Registros jerarquizados.

Definición: Tipo de dato estructurado estático en el que sus componentes (llamados campos) pueden ser de distinto tipo.

	Homogénea	*Heterogénea*
Estática	Tablas	Registros
Dinámica	Tablas dinámicas	Memoria Dinámica

Declaración Tipo de Datos…

```
struct nombre_registro {
    tipo  nombre_campo1;
    tipo  nombre_campo2;
    . . .
    tipo  nombre_campoN;
} ;
```

Otra forma:

```
typedef struct {
    tipo  nombre_campo1;
    tipo  nombre_campo2;
    . . .
    tipo  nombre_campoN;
} nombre_registro ;
```

Declaración de registros.

```
//Declaración del tipo alumno
struct alumno
{
    int tlf;
    int edad;
    float peso;
    char sexo;
    int DNI;
    char letraNIF;
};

int main()
{
  //Declaración de variables
  alumno a, b;
  ...
  return 0;
}
```

```
//Declaración de tipos
struct persona
{
    float peso;
    double altura;
};

struct fecha {
    int dia;
    int mes;
    int anio;
};

int main()
{
  //Declaración de variables
  persona p1, p2;
  fecha  f;
  ...
  return 0;
}
```

Operaciones con registros.

```
#include <iostream>
using namespace std;
struct punto {
  int x;
  int y;
  };
int main ( ){
  punto uno, dos;
  punto origen ={0,0};
  uno.x = 3;
  uno.y = 4;
  //cin >> dos;   Error
  cin >> dos.x;
  cin >> dos.y;
  origen = dos;
  //cout << origen;  Error
  cout << "Origen tiene de coordenada x : "
       << origen.x <<"\n";
  cout << "Origen tiene de coordenada y : "
       << origen.y;
  return 0;
}
```

Tipos de Operaciones
Declaración del tipo punto.
Declaración de variables registros. Declaración e inicialización. Asignación de valores a sus campos.
** Inicialización desde teclado.
Copia de registros completos.
** Muestra por pantalla el contenido de los campos del registro.
**** No se pueden leer desde teclado ni mostrar en pantalla registros completos, se hace campo a campo.**

Registros jerarquizados.

```
#include <iostream>
using namespace std;
struct  fechas{
  int   dia;
  int   mes;
  int   anio;
  };
struct  InforPersona{
  float     peso;
  float     altura;
  fechas    nacimiento;
  };
int main () {
  InforPersona persona;
  cout << "Indica tu peso: ";
  cin >> persona.peso;
  cout << "Indica el dia del mes en que naciste: ";
  cin >> persona.nacimiento.dia;
  cout << "Indica el numero del mes en que naciste: ";
  cin >> persona.nacimiento.mes;
  persona.nacimiento.anio = 1995;
  cout << " Naciste el dia " << persona.nacimiento.dia << " mes "
    << persona.nacimiento.mes << " a" << char(164) <<"o "
    << persona.nacimiento.anio << " y tu peso es : " << persona.peso << " Kg.";
  return 0;
}
```

2. Tablas. Tablas de registros.

Definición: Tipo de dato estructurado estático en el que sus componentes (llamados elementos) tienen que ser del mismo tipo.

	Homogénea	*Heterogénea*
Estática	Tablas	Registros
Dinámica	Tablas dinámicas	Memoria Dinámica

Sintaxis declaración de una tabla (array, vector):

Tipo_cada_elemento nombre_variable [número_de_elementos];

```
#include <iostream>
using namespace std;
int main () {
  float notas[10];        //Vector de 10 reales
  char palabra[20];       //Vector de 20 caracteres
  double coleccion[700];  //Vector de 700 reales de doble precisión
  . . .
  return 0;
}
```

Acceso a los elementos de una tabla.

```
int main () {      //Cada elemento de un vector tiene un índice.
   int iva[45];    //El primer elemento tiene como índice 0 y el último el 44,
                   //es decir el número de elementos menos uno.
   iva[0] = 22;    //El primer elemento del vector contiene el valor 22
   iva[44] = 8;    //El último elemento del vector contiene el valor 8
   iva[45] = 13;   //Error índice 45 incorrecto fuera de rango
   . . .
}
```

Dimensiones de un tabla.

Una tabla puede tener una o más dimensiones. Para cada dimensión hay que indicar el tamaño o número de elementos. Sintaxis general de declaración de una tabla:

Tipo_elemento nombre_tabla[Tama D1] [Tama D2] [Tama D3]. . . [Tama DN];

<u>Ejemplos:</u>

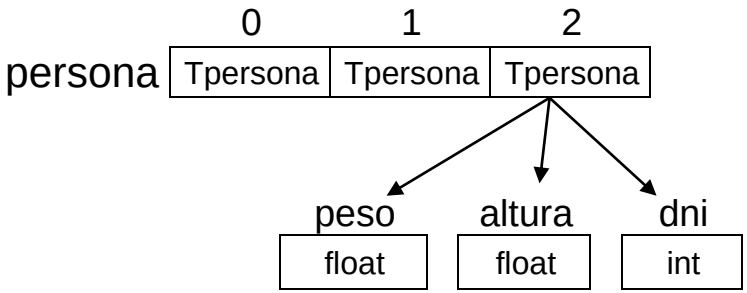
```
int tab[10];                  //Una dimensión (10 elementos)
char palabras[10][20];        //Dos dimensiones (10x20 elementos)
float almacen[10][15][10];    //Tres dimensiones (10x15x10 elementos)
```

Operaciones con tablas:

```
#include <iostream>
using namespace std;
int main (){
  int tab1[5];
  int tab2[3][3];
  int tab3[5] = { 2, 4, 6, 8, 10};
  tab1[0] = 3;
  tab2[1][0] = 5;
  cin >> tab3[0];
  //tab1 = tab3;   ERROR
  for (int i=0; i < 5; i++)
    tab1[i]= tab3[i];
  //cin >> tab3    ERROR
  for (int i=0; i < 5; i++)
    cin >> tab3[i];
  for (int i=0; i < 3; i++)
    for (int j=0; j < 3; j++)
        cin >> tab2[i][j];
  cout << tab3[0];
  cout << tab2[0][0];
  //cout << tab3; ERROR
  cout << "Muestro la tabla 1: ";
  for (int i=0; i < 5; i++)
    cout << tab3[i] << " ";
  cout << "\nMuestro la tabla 2: ";
  for (int i=0; i < 3; i++)
    for (int j=0; j < 3; j++)
        cout << " " << tab2[i][j];
  return 0;
}
```

Tipos de Operaciones
Declaración de tablas tab1 y tab2.
Declaración e inicialización de tab3. Asignación de valores. Inicialización desde teclado. **
Copiar una tabla en otra.
Leer desde teclado una tabla
Mostrar por pantalla un elemento de una tabla. **
Mostrar una tabla por pantalla.
**** En general no se pueden leer desde teclado ni mostrar en pantalla tablas completas, se ha de hacer elemento a elemento por cada dimensión. **** La única excepción son los vectores de caracteres (cadenas).**

Tablas de registros.




```
#include <iostream>
using namespace std;
struct  Tpersona{
  float     peso;
  float     altura;
  int       dni;
  };
int main () {
  Tpersona persona[3];
  for (int i=0; i <3; i++){ //Relleno la tabla con valores
    cout << "Introduce el peso de la persona " << i <<": ";
    cin >> persona[i].peso;
    cout << "Introduce la altura de la persona " << i << ": ";
    cin >> persona[i].altura;
    cout << "Introduce el dni (sin letra) de la persona " << i << ": ";
    cin >> persona[i].dni;
  }
  for (int i=0; i <3; i++){ //Muestro la tabla por pantalla
    cout << "\nPersona n." << i << ":\n";
    cout << "\tpeso: " << persona[i].peso;
    cout << "\taltura: " << persona[i].altura;
    cout << "\tdni: " << persona[i].dni;
  }
  return 0;
}
```

3. Esquemas de recorrido y búsqueda.

El **esquema de recorrido** se utiliza cuando **obligatoriamente se tenga** que tratar **todos** los elementos de una estructura de datos.

Esquema:

```
tipo tabla [ N_Elementos ];
for (int i=0; i < N_Elementos, i++)
    Tratamiento del elemento_i;
Tratamiento final
```

Rellenar la tabla con valores leídos desde teclado:

```
int tabla1[6];
for (int i=0; i <6; i++)
    cin >> tabla1[i];
```

Mostrar por pantalla la tabla completa

```
for (int i=0; i <6; i++)
    cout << tabla1[i];
```

Rellenar la tabla con el valor de su índice:

```
for (int i=0; i <6; i++)
    tabla1[i] = i;
```

El **esquema de búsqueda** se utiliza cuando **no siempre** se tienen que tratar todos los elementos de una estructura de datos.

Esquema:

```
Accedo al primer elemento.
No encontrado.
while ( no último && no encontrado )
    if  (elemento_i == buscado)  encontrado
    else  accedo al siguiente elemento
if (encontrado)
   tratar elemento_encontrado
else
   tratar ausencia_del_elemento
```

Buscar la letra a en un texto leído desde teclado:

```
char letras[20];
cin >> letras;
int i=0;
bool encontrado = false;
while ( i<20 && encontrado == false )
  if (letras[i] == 'a') encontrado = true;
  else  i++;
if (encontrado)
  cout << "Está la a";
else
  cout << "NO está la a";
```

Ejemplos de problemas de recorrido o de búsqueda	Esquemas	
	de recorrido	de búsqueda
¿Cuántos “Luis” hay en la tabla?	X	
¿Cuál es el valor máximo almacenado?	X	
¿Esta “Juan” en la tabla?		X
¿Cuánto suman los valores positivos almacenados?	X	
¿En qué fila y columna está almacenado “Javier”?		X
Cargar la tabla completa con valores leídos desde teclado	X	
¿Son idénticas dos tablas en sus elementos?		X
Indique por pantalla si una matriz es simétrica		X
Copiar una tabla en otra	X	

4. Cadenas.

Una cadena es una tabla de una dimensión de elementos de tipo **char** acabada con el código ASCII número 0 ('\0').

```
char pal1[120];
```

Operaciones con cadenas:

```
cin  >> pal1; /*Lee una cadena de caracteres hasta encontrar un
              espacio, tabulador o retorno de carro. Si se
              introduce por teclado Juan Luis, pal1 solo tendrá
              la palabra Juan*/
cin.getline(pal1,120); /*Lee una cadena formada por múltiples
              palabras con un tamaño máximo de 120 caracteres*/
cout << pal1; /*Mostrará por pantalla cada elemento de la tabla
              hasta encontrar el código ASCII número 0 '\0'*/
```

Operaciones con cadenas:

C define un conjunto de operaciones para el tratamiento de cadenas en las librerías **cstring** o **string.h**

Supongamos que c1 y c2 son dos cadenas de caracteres y car un carácter.

Nombre de la Función	***Actuación***
strcpy(c1, c2)	Copia c2 en c1
strcat(c1, c2)	Concatena c2 al final de c1
strlen(c1)	Devuelve la longitud de c1
strcmp(c1, c2)	Devuelve 0 si son iguales, menor que 0 si c1<c2 y mayor que 0 cuando c1>c2
strchr(c1, car)	Devuelve un índice a la primera ocurrencia de car en c1
strstr(c1, c2)	Devuelve un índice a la primera ocurrencia de c2 en c1

```
#include <iostream>
#include <cstring>
using namespace std;

#define M 3
typedef char cadena[30];

int main()
{
    cadena tabla[M], leida;
    bool encontrado;
    int i;

    for (i=0; i<M; i++)
    {
        cout << "\nIntroduce la palabra " << i+1 << ": ";
        cin >> tabla[i];
    }
    cout << "Introduce la palabra a buscar: ";
    cin >> leida;

    i = 0;
    encontrado = false;
    while ( i < M && encontrado == false )
        if (strcmp(tabla[i], leida) == 0)
            encontrado = true;
        else
            i++;

    if (encontrado) cout << "Se encontró en la posición "
                    << i << endl;
    else    cout << "No se encontró.";
    return 0;
}
```

Tipos de Operaciones

Utilización de typedef

Tabla de 3 cadenas, cada una de 30 caracteres.

Recorrido, rellenando la tabla de 3 palabras desde teclado.

Palabra a buscar

Búsqueda de la palabra en la tabla.
Utilización de strcmp

Nota. No se puede comparar dos vectores con un operador relacional ni se puede asignar un vector a otro.

if (tabla[i]==leida) ERROR